Andromeda Spec 09: Load/Store Unit

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `lsu` · **Milestone:** Load/store unit and memory interface

1. What this unit does

The LSU executes every load and store on port P3, integer and FP. It computes addresses, keeps memory accesses correct even though they run out of order, and owns the data memory port. It contains an 8-entry load queue (LQ) and an 8-entry store queue (SQ).

The key rule: **stores write memory only after they commit.** A flush therefore never has to undo a memory write.

2. How it works

2.1 The ordering problem

Loads and stores are issued out of order, like everything else. That creates two hazards:

- A load may execute before an older store to the same address has even computed its address.
- Even when the store executed first, it hasn't written memory yet (it waits for commit), so memory still holds the old value.

Either way, a load that just read memory would get stale data. The rule the LSU enforces is: **every load must return the value it would get if all instructions ran in program order.** Loads may still execute out of order with each other and ahead of younger stores; they only have to see every older store and no younger one.

2.2 The store queue

Each store gets an SQ entry at dispatch, in program order, even before it executes. The entry starts as "address unknown". When the store issues, it writes its address, data and byte mask into its entry and reports completion to the ROB (it has no register result). When the ROB commits it, the entry is marked committed. Committed entries drain from the SQ head to memory in order, one per cycle.

2.3 How a load finds older stores

At dispatch, each load records the SQ tail position as its `sq_id`. `sq_id` only increments and therefore indicates the store insn age. That snapshot means "every store before this point is older than me". It travels with the load, so when the load finally executes, it checks only the stores from the SQ head up to its marker, never stores that arrived later. Committed stores still waiting to drain are in that range too, so they are checked as well. Store queue id and entry is allocated at dispatch in program order, so when load arrives at LSU, the stores before it would already have allocated entries.

For the older stores in range, starting from the youngest:

Case	Load does

Case	Load does
Any older store's address is still unknown	Waits in its LQ entry and retries later
Youngest matching older store fully covers the load	Takes the data from the store queue (forwarding); no memory access
A matching older store only partly overlaps	Waits until that store drains to memory
No older store matches	Reads memory

The four situations happen because loads and stores can happen out of order. Case 1 happens when an older store is issued after younger load. Case 2 happens when older store is issued before younger load. Case 3 is when the store address range and load address overlap. For instance, store half word at 0x2 and load word at 0x0.

2.4 Example

Assume x6 = x8 = address A.

```
1: sw  x5, 0(x6)     # store to A; x5 comes from a slow divide
2: lw  x7, 0(x8)     # load from A: must get x5
3: sw  x9, 0(x8)     # younger store to A
4: lw  x10, 4(x8)    # load from A+4: unrelated
```

Instr	Gets at dispatch	Older stores it must check
1 sw	sq0	n/a
2 lw	lq0, marker 1	sq0
3 sw	sq1	n/a
4 lw	lq1, marker 2	sq0, sq1

- Store 1 can't issue yet (x5 isn't ready), so sq0's address is unknown. Load 2 issues anyway, sees an unknown older store in its range and waits in lq0.
- Store 3 issues and fills sq1 with address A. Load 2 ignores it: sq1 is outside its range.
- The divide finishes and store 1 fills sq0. Load 2 retries, finds sq0 covers it exactly, and forwards x5.
- Load 4 checks sq0 and sq1, finds neither touches A+4, and reads memory with `dmem_req_id = 1`. The response comes back tagged 1, so it goes to lq1.

2.7 Other behaviour

- **Address and alignment.** The address is base + imm. Misaligned halfword or word accesses raise address-misaligned exceptions (cause 4 for loads, 6 for stores); a load fault from memory (`dmem_resp_err`) raises a load access fault (cause 5). Stores reach memory only after commit, too late to trap precisely, so a store access fault (cause 7) must be detected from the address at execute, for example with a simple address-range check; memory-side store errors are out of scope for v1. All are reported through `wb_t` with `exc_tval` = the address.
- **Sizes and extension.** Loads sign- or zero-extend by size. FLW writes the FP file and FSW takes its data from the FP file.

- **Port sharing.** The memory port is shared: committed stores win when the SQ is more than half full, otherwise loads go first.
- **Flush.** The LQ and uncommitted SQ entries are cleared. Committed stores stay and keep draining, since they are already architecturally done.
- **FENCE.** Waits at the ROB head until `sq_empty`, so all earlier stores have reached memory.
- **Side effects.** Loads execute speculatively, so v1 assumes reads have no side effects. An uncached I/O region where loads wait for the ROB head is an open item.

3. Interface

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`
`in_valid` / `in_uop`	in	1 / uop_t	From rr_stage (op, size, signed, imm, rob_id, prd; `lq_id` and the `sq_id` marker for loads, `sq_id` for stores)
`in_base` / `in_sdata`	in	32 / 32	Base address register and store data
`in_ready`	out	1	Can accept a uop this cycle
`lq_alloc_valid` / `sq_alloc_valid`	in	4 / 4	Allocation from dispatch
`lq_alloc_id` / `sq_alloc_id`	out	4 × 4 / 4 × 4	Allocated indices with wrap bit (tail + n)
`sq_tail`	out	4	Current SQ tail, for load markers
`lq_free_cnt` / `sq_free_cnt`	out	4 / 4	Free entries
`cm_load_cnt` / `cm_store_cnt`	in	3 / 3	Loads and stores retired this cycle: frees LQ entries, marks SQ entries committed
`sq_empty`	out	1	No committed stores left to drain (used by FENCE)
`wb`	out	wb_t	Load result or store completion, including exceptions
`wakeup`	out	wakeup_t	Load destination tag
`dmem_req_valid` / `dmem_req_ready`	out / in	1 / 1	Request handshake
`dmem_req_we` / `dmem_req_addr` / `dmem_req_wdata` / `dmem_req_wstrb` / `dmem_req_id`	out	1 / 32 / 32 / 4 / 3	Request fields; `dmem_req_id` is the LQ index without its wrap bit
`dmem_resp_valid` / `dmem_resp_rdata` / `dmem_resp_id` / `dmem_resp_err`	in	1 / 32 / 3 / 1	Load response; stores get no response

The memory side is a simple valid/ready protocol so it can sit in front of SRAM, a cache or an adapter to a bus such as Wishbone.

4. Implementation notes

- **Pipeline.** A reasonable split: cycle 1 computes the address and checks alignment; cycle 2 searches the SQ and either forwards, sends the memory request or parks the load in the LQ. Stores spend cycle 2 writing their SQ entry.
- **SQ search.** Compare the load's word address against all 8 SQ entries in parallel (a small CAM), mask by "in range" (head to marker) and "address known", then pick the youngest match before the marker. The range mask is easy to build from the head and marker pointers with their wrap bits.
- **Byte masks.** Store each SQ entry's address as a word address plus a 4-bit byte mask. "Fully covers" means the load's mask is a subset of the store's mask in the same word; "partial overlap" means the masks intersect but aren't a subset. Forwarded data must be shifted to the load's byte offset before extension.
- **Retrying parked loads.** When a store writes its SQ entry or drains, parked loads may become able to proceed. The simplest scheme is to retry the oldest parked load each cycle the pipeline is free. Smarter wakeup can come later.
- **Store data timing.** v1 issues a store only when both base and data are ready. Splitting address and data (so the address is known earlier and loads wait less) is a possible later optimization.

5. Verification checklist

- ☐ Unit test against a software memory-ordering model with random loads and stores over a small set of addresses and all sizes
- ☐ Load issued before an older store's address is known waits, then gets the right value
- ☐ Younger stores never forward to older loads
- ☐ Forwarding for every size and byte offset combination; partial overlaps wait
- ☐ Committed but undrained stores are checked by new loads after their ROB entries are reused
- ☐ Queue pointers across many wraparounds
- ☐ Flush with committed stores still draining: they complete, uncommitted ones vanish
- ☐ Misaligned and faulting accesses raise the right causes with the right `exc_tval`
- ☐ Memory model with random latency and out-of-order responses